Computer Hardware Basics: Beginner Module Research Notes (as of Oct 2026)

Source-access note: BBC Bitesize (bbc.co.uk) blocks this research tool, the Microchip product page returned HTTP 403, and the PDF datasheets could not be text-extracted. ATmega328P figures therefore come from Arduino's official store page plus Wikipedia, which cites the Atmel/Microchip datasheet. They agree with each other and with search snippets of the datasheet. Before publishing, a person should check them once against the Microchip datasheet (DS40002061B): https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf

1. Main parts of a computer: CPU, RAM, storage, I/O (beginner-safe definitions)

Takeaway

A computer takes **input**, **processes** it in the CPU, holds working data and running programs in fast **volatile RAM**, keeps files long-term in **non-volatile storage** (SSD/HDD/flash), and produces **output**. Beginners often confuse RAM (memory) with storage.

Cited Findings

- RAM is volatile. It needs a steady power supply because it keeps refreshing the bits it stores, so a power interruption wipes its contents. SSDs and hard drives are non-volatile and keep data without power. — Corsair: Volatile vs non-volatile memory (commercial vendor source, but the content matches the standard definition)
- RAM is much faster than storage but costs more per GB. That is why systems pair a small amount of RAM (typically 16–32 GB in consumer PCs) with a much larger SSD/HDD (typically 1–2 TB). — Corsair
- The CPU contains a **control unit (CU)**, which decodes instructions and sends control signals, and an **ALU**, which does arithmetic, logic and comparisons. It also contains registers such as the PC, MAR, MDR and CIR. — Save My Exams (CIE IGCSE): The Fetch-Decode-Execute Cycle
- Example of a modern single-board computer: the Raspberry Pi 5 has a Broadcom BCM2712 quad-core 64-bit Arm Cortex-A76 CPU at 2.4 GHz and 1/2/4/8/16 GB of LPDDR4X SDRAM. Its storage is a microSD card, or an NVMe SSD over a PCIe 2.0 x1 interface. It runs an operating system (Raspberry Pi OS "Trixie"; legacy "Bookworm"). — Raspberry Pi 5 product page; Raspberry Pi 5 product brief (Apr 2026); 16GB Pi 5 news
- I/O on a microcontroller board: the Arduino Uno R3 has 14 digital I/O pins (6 can do PWM output) and 6 analog inputs. — Arduino UNO R3 docs; Arduino Store UNO R3

Inferences

- Suggested beginner definitions, synthesized from the sources above:
  - **CPU (processor):** the part that runs a program's instructions one at a time, using the CU to coordinate and the ALU to calculate.
  - **RAM (main memory):** fast working memory that holds the programs and data in use right now. It is volatile, so it is emptied when power is lost.

    - **Storage (secondary storage):** slower, larger memory that keeps files and programs when the power is off (non-volatile). Examples are SSD, HDD, microSD/USB flash.
    - **Input devices** send data into the computer (keyboard, mouse, microphone, sensor, button). **Output devices** present results (screen, speaker, printer, LED, motor).
- A useful analogy (flag it to students as an analogy): RAM is your desk (fast, small, cleared at the end of the day), storage is the filing cabinet (big, slower, permanent).
- Beware the misconception "my phone has 256 GB of memory". That figure is storage. RAM is the separate, smaller number (e.g., 8 GB).

Gaps

- No university or BBC Bitesize definition page could be fetched (Bitesize blocked). The Corsair source is commercial. The team may want to cite a textbook, e.g. Nisan & Schocken, *The Elements of Computing Systems* (MIT Press), referenced at nand2tetris.org.
- The ROM/firmware/BIOS distinction was not researched.

2. Fetch–decode–execute cycle (PC, IR, memory, ALU, CU) with a LOAD/ADD/STORE example

Takeaway

The CPU repeats three steps. **Fetch** copies the instruction at the address held in the program counter (PC) from memory into the instruction register (IR/CIR), and the PC is incremented. **Decode** is the control unit working out the opcode and operand. **Execute** carries the instruction out, e.g. the ALU adds or data moves to or from memory. A jump/branch overwrites the PC.

Cited Findings

- **PC** holds the address of the next instruction to be executed. At the start of fetch, that address is copied into the **Memory Address Register (MAR)**. — Save My Exams (CIE O Level); Save My Exams (IGCSE)
- The instruction at that address travels over the data bus into the **Memory Data Register (MDR)** and is then copied into the **Current Instruction Register (CIR)** to wait for decoding. — Save My Exams (IGCSE)
- The PC is incremented during fetch so that it points to the next instruction. — Save My Exams (IGCSE)
- **Decode:** the control unit splits the CIR contents into an **opcode** (which operation) and an **operand** (which data or address). — Save My Exams (IGCSE)
- **Execute:** the CU sends signals to the relevant components. The ALU may calculate or compare, data may be transferred, or a branch instruction may change the PC. — Save My Exams (IGCSE); also Computer Science Wiki: machine instruction cycle
- Some syllabuses (e.g., OCR) describe a separate fourth "store"/write-back step. Wording differs between exam boards. — TutorChase OCR GCSE notes
- University-level reference on the same cycle: UMD CMSC311 notes: Fetch, Decode, Execute (fetch failed, cited from search listing only); UVM CS2210 (Cafiero) fetch-decode-execute (page truncated, contents not verified)

Inferences

- **Worked example.** This is a constructed teaching example on a hypothetical accumulator machine, not a real instruction set. It is consistent with the cycle described above. Program in memory: address 0

`LOAD 10`, address 1 `ADD 11`, address 2 `STORE 12`. Data: address 10 = 5, address 11 = 3.

1. PC=0. Fetch `LOAD 10` into the IR, and PC becomes 1. Decode: opcode LOAD, operand address 10. Execute: read memory[10]=5 into the accumulator (ACC=5).
2. PC=1. Fetch `ADD 11`, and PC becomes 2. Decode: ADD, address 11. Execute: read memory[11]=3, the ALU computes 5+3, ACC=8.
3. PC=2. Fetch `STORE 12`, and PC becomes 3. Decode: STORE, address 12. Execute: write ACC (8) to memory[12].

- Tell students that "incrementing the PC" means moving to the next instruction. On real CPUs that step can be more than 1 address, because instructions can be several bytes long. AVR instructions, for example, are 16 or 32 bits wide. Flag that last fact as unverified here: it is common knowledge, but no datasheet text was fetched for it.
- Real CPUs overlap these stages (pipelining). Beginner material should present the cycle as sequential but note that it is simplified.

Gaps

- Bitesize and Isaac Computer Science pages were not retrievable. The definitions here come from exam-revision sites (Save My Exams, TutorChase), which are reliable for terminology but secondary.

3. Bits, bytes, value ranges, KB vs KiB, and logic gates (AND/OR/NOT)

Takeaway

A **bit** is one binary digit (0 or 1). A **byte** is 8 bits. *n* bits give 2^n patterns, so 4 bits represent 0–15 and 8 bits represent 0–255 (unsigned). Officially **kB = 1000 bytes** and **KiB = 1024 bytes**. AND, OR and NOT gates combine bits following simple truth tables.

Cited Findings

- 1 byte (B) = 2^3 bits = 8 bits. — NIST: Prefixes for binary multiples
- In December 1998 the IEC approved binary prefixes. kibi (Ki) = 2^10 = 1024, mebi (Mi) = 2^20 (1 MiB = 1,048,576 B), gibi (Gi) = 2^30 (1 GiB = 1,073,741,824 B). The SI "kilo" means 1000 (1 kbit = 1000 bit). Using "kilo" informally to mean 1024 caused confusion. — NIST
- AND outputs 1 only when both inputs are 1. OR outputs 1 when at least one input is 1. NOT inverts its single input. — Save My Exams AQA GCSE: Truth tables; TutorChase CIE A-Level: Truth tables for logic gates
- Truth tables (standard):

A	B	A AND B	A OR B
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

A	NOT A
0	1

A	NOT A
1	0

- Nand to Tetris builds "a general-purpose computer system and a modern software hierarchy from the ground up", starting from NAND gates. The free materials accompany Nisan & Schocken, *The Elements of Computing Systems* (MIT Press), and a paid Coursera version exists. — nand2tetris.org

Inferences

- Value ranges follow from arithmetic: 2^4 = 16 patterns, giving 0–15 (0000–1111); 2^8 = 256 patterns, giving 0–255 (00000000–11111111). A common beginner error is to say 8 bits go up to 256. The count of values is 256, but the maximum is 255 because counting starts at 0.
- Common misconceptions to address:
  - "1 KB = 1024 bytes" is officially KiB.
  - Drive makers use decimal units while some operating systems show binary units, so a "1 TB" drive shows about 931 GiB. Flag this: it is derived arithmetic (10^12 / 2^30 ≈ 931.3), not fetched from a source.
  - Lowercase b means bits and uppercase B means bytes, as in internet speeds in Mbit/s versus file sizes in MB.
- Hook to motivate the topic: NAND alone can build every other gate (it is functionally complete), which is the premise of Nand to Tetris. The site title implies this; no explicit page text about it was fetched.

Gaps

- No primary university source was fetched for the gate truth tables. They are standard, and the revision sources agree.

4. Microcontroller (ATmega328P / Arduino Uno) vs PC/single-board computer; GHz misconception

Takeaway

A microcontroller puts a CPU, flash program memory, SRAM, EEPROM and I/O on **one chip**. The ATmega328P on the Arduino Uno has **32 KB flash (about 0.5 KB used by the bootloader), 2 KB SRAM and 1 KB EEPROM**, and runs at **16 MHz** on the Uno. It runs one program with no operating system. A PC or Raspberry Pi has a separate CPU, gigabytes of RAM, separate storage and an operating system. Clock speed (GHz) alone does not determine performance.

Cited Findings

- Arduino UNO R3: microcontroller ATmega328P; flash 32 KB, of which 0.5 KB is used by the bootloader; SRAM 2 KB; EEPROM 1 KB; clock speed 16 MHz; operating voltage 5 V; 14 digital I/O (6 PWM); 6 analog inputs. — Arduino Store: UNO R3
- The Uno's 16 MHz clock comes from a ceramic resonator. EEPROM is 1 KB and is not erased when the power is off. — Arduino docs: UNO R3
- ATmega328P chip: 8-bit RISC core with a **modified Harvard architecture**; 32 KB in-system-programmable flash with read-while-write; 2 KB SRAM; 1 KB EEPROM; up to 20 MHz / 20 MIPS;

operates from 1.8–5.5 V; up to 23 GPIO lines. Its best-known use is the Arduino Uno. — Wikipedia: ATmega328 (cites the Atmel ATmega328/P datasheet)

- Datasheet endurance: flash is rated for 10,000 write/erase cycles and EEPROM for 100,000. — search snippet of Atmel-7810 ATmega328P datasheet (snippet only, not full text; verify)
- Maximum clock: the chip itself is rated up to 20 MHz (depending on supply voltage), but the Uno board runs it at 16 MHz. The older Atmel datasheet says "up to 16 MIPS at 16 MHz", and newer Microchip material says 20 MHz. Both fit the throughput of roughly 1 MIPS per MHz. — Wikipedia: ATmega328; Arduino Store
- Comparison point, Raspberry Pi 5: 2.4 GHz quad-core 64-bit CPU, up to 16 GB RAM, microSD/NVMe storage, runs Raspberry Pi OS. — Raspberry Pi 5
- Why GHz is not everything: instructions per second ≈ instructions per cycle (IPC) × clock rate, and IPC varies by design and workload. — Wikipedia: Instructions per cycle. For example, a 3 GHz Core 2 Duo is much faster than a 3 GHz Pentium D because it does more work per clock. — Android Authority: Gary explains IPC
- Programs not written to use multiple cores gain nothing from extra cores. Cache size matters too. CPUs boost their speed and "throttle" down when hot. — Tom's Hardware: What is clock speed

Inferences

- How the Uno maps onto the general model (synthesized from the sources above):
  - The CPU is the AVR core.
  - Flash holds the program, like "storage", and is non-volatile.
  - SRAM holds variables while running, like "RAM", and is volatile.
  - EEPROM is a small non-volatile area for settings.
  - The I/O pins act as the input and output devices.
- "Modified Harvard" means program memory (flash) and data memory (SRAM) are separate memories with separate paths. A PC, by contrast, loads programs from storage into the same RAM that holds data (von Neumann style at the programmer's level). This is general knowledge; datasheet text describing it was not fetched.
- Key differences for beginners:
  - Scale: the Uno has 2 KB of SRAM, the Pi 5 has up to 16 GB of RAM, about 8 million times more. That figure is derived arithmetic: 16 GiB ÷ 2 KiB = 2^23 ≈ 8.4 million.
  - The Uno has no OS and runs one sketch from power-on. The Pi and a PC boot an OS that loads many programs.
  - The Uno's 16 MHz is about 150× slower in clock than the Pi's 2.4 GHz, but that is the right trade-off for simple, real-time, low-power control tasks.
- Arduino's documentation spells EEPROM as "1kb". That means 1 KB (kilobyte), not 1 kilobit. Worth noting given the KB/kb lesson in section 3.

Gaps

- The Microchip product page (403) and the datasheet PDF text could not be read directly. The memory sizes are confirmed by the Arduino store and Wikipedia plus datasheet snippets, and are consistent across all sources. Do one manual check against Microchip DS40002061B before publishing.
- The Arduino memory guide (https://docs.arduino.cc/learn/programming/memory-guide/) returned no content and was not used.
- Speed grade versus voltage (e.g., 0–20 MHz at 4.5–5.5 V) was not verified.